AI ASSISTED CODING

ASSIGNMENT -9.5

Name: Hari Priya

H.T.No: 2303A51104

Problem 1: String Utilities Function

Consider the following Python function:

```
def reverse_string(text):
    return text[::-1]
```

Task:

1. Write documentation in:

o (a) Docstring

o (b) Inline comments

o (c) Google-style documentation

2. Compare the three documentation styles.

3. Recommend the most suitable style for a utility-based string library.

```python
# Docstring documentation:
def reverse_string(text):
    """
    This function takes a string as input and returns the reverse of that string.
    """
    return text[::-1]
# Example usage:
input_string = "Hello, World!"
reversed_string = reverse_string(input_string)
print(f"Original string: {input_string}")
print(f"Reversed string: {reversed_string}")

# Inline documentation:
def reverse_string(text):
    # This function takes a string as input and returns the reversed version of that string.
    return text[::-1]
input_string = "Hello, World!"
reversed_string = reverse_string(input_string)
print(f"Original string: {input_string}")
print(f"Reversed string: {reversed_string}")
```

```python
# Google style documentation:
def reverse_string(text):
    """
    Reverses the given string.
    Args:
        text (str): The string to be reversed.
    Returns:
        str: The reversed string.
    Raises:
        TypeError: If the input is not a string.
    """
    if not isinstance(text, str):
        raise TypeError("Input must be a string")
    return text[::-1]
input_string = "Hello, World!"
reversed_string = reverse_string(input_string)
print(f"Original string: {input_string}")
print(f"Reversed string: {reversed_string}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

```
PS C:\Users\HARI PRIYA\Desktop\AIAC> python -m pydoc lab_9_5
Original string: Hello, World!
Reversed string: !dlroW ,olleH
Original string: Hello, World!
Reversed string: !dlroW ,olleH
Original string: Hello, World!
Reversed string: !dlroW ,olleH
Help on module lab_9_5:
```

```
NAME
    lab_9_5 - # Docstring documentation:


FUNCTIONS
    reverse_string(text)
        Reverses the given string.
        Args:
            text (str): The string to be reversed.
        Returns:
            str: The reversed string.
        Raises:
            TypeError: If the input is not a string.

DATA


DATA
    input_string = 'Hello, World!'
    reversed_string = '!dlroW ,olleH'

FILE
    c:\users\hari priya\desktop\aiac\lab_9_5.py
```

Docstring: Gives a short description inside the function. It is clear and commonly used for simple explanations.

Inline comments: Very brief and placed above the code, but not structured or detailed.

Google style documentation: Well-structured with sections like Args, Returns, and Raises. It is detailed and professional.

For a utility-based string library, the Google style documentation is most suitable because it is clear, organized, and helpful for other developers.

Problem 2: Password Strength Checker

Consider the function:

def check_strength(password):

return len(password) >= 8

Task:

1. Document the function using docstring, inline comments, and

Google style.

2. Compare documentation styles for security-related code.

3. Recommend the most appropriate style.

```python
# Docstring documentation:
def check_strength(password):
    """
    This function checks the strength of a given password based on certain criteria.

    Args:
        password (str): The password to be evaluated.

    Returns:
        str: A                                    ngth of the password (e.g., "Weak", "Moderate", "Strong").
    """
    return len(password) >= 8
# Example usage:
input_password = "P@ssw0rd"
is_strong = check_strength(input_password)
print(f"Password: {input_password}")
print(f"Is the password strong? {'Yes' if is_strong else 'No'}")

# Inline documentation:
def check_strength(password):
    # This function checks if the password is strong based on its length.
    return len(password) >= 8
input_password = "P@ssw0rd"
is_strong = check_strength(input_password)
print(f"Password: {input_password}")
print(f"Is the password strong? {'Yes' if is_strong else 'No'}")
```

(parameter) password: Any

password : *str*
The password to be evaluated.

```python
# Google style documentation:
def check_strength(password):
    """
    Evaluates the strength of a password based on its length.

    Args:
        password (str): The password to be evaluated.

    Returns:
        bool: True if the password is strong (length >= 8), False otherwise.
    Raises:
        TypeError: If the input is not a string.
    """
    if not isinstance(password, str):
        raise TypeError("Input must be a string")
    return len(password) >= 8
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

```
PS C:\Users\HARI PRIYA\Desktop\AIAC> & "C:\Users\HARI PRIYA\AppData\Local\Microsoft\WindowsApps\python3.11.exe" "c:/Users/HARI PRIYA/Desktop/AIAC
9_5.py"
Password: P@ssw0rd
Is the password strong? Yes
Password: P@ssw0rd
Is the password strong? Yes
```

Docstring style gives a clear description, arguments, and return values. It is readable but may lack strict structure for larger security projects.

Inline documentation is short and simple, but not detailed enough for security-related code where clarity is very important.

Google style documentation is more structured and detailed. It clearly explains arguments, return types, and possible errors which is important in security code.

For security-related code, Google style documentation is the most appropriate because it is clear, structured, and explains exceptions properly, reducing misunderstanding and security risks.

Problem 3: Math Utilities Module

Task:

1. Create a module math_utils.py with functions:

o square(n)

o cube(n)

o factorial(n)

2. Generate docstrings automatically using AI tools.

3. Export documentation as an HTML file.

```python
def square(n: float) -> float:
    """
    This function takes a number as input and returns its square.

    Parameters:
    n (float): The number to be squared

    Returns:
    float: The square of the input number
    """
    return n ** 2
def cube(n: float) -> float:
    """
    This function takes a number as input and returns its cube.

    Parameters:
    n (float): The number to be cubed

    Returns:
    float: The cube of the input number
    """
    return n ** 3
def factorial(n: int) -> int:
    """
    This function takes a number as input and returns its factorial.

    Parameters:
    n (int): The number to be factorialized

    Returns:
    int: The factorial of the input number
    """
    if n == 0:
        return 1
    else:
        return n * factorial(n-1)
```

```
PS C:\Users\HARI PRIYA\Desktop\AIAC> python -m pydoc lab
Help on module lab:

NAME
    lab

FUNCTIONS
    cube(n: float) -> float
        This function takes a number as input and returns its cube.

        Parameters:
        n (float): The number to be cubed

        Returns:
        float: The cube of the input number

    factorial(n: int) -> int
        This function takes a number as input and returns its factorial.

        Parameters:
        n (int): The number to be factorialized

        Returns:
        int: The factorial of the input number

    square(n: float) -> float
        This function takes a number as input and returns its square.

        Parameters:
        n (float): The number to be squared

        Parameters:
        n (float): The number to be squared

        n (float): The number to be squared


        Returns:
        float: The square of the input number


FILE
FILE
    c:\users\hari priya\desktop\aiac\lab.py
```

index
lab c:\users\hari priya\desktop\aiac\lab.py

Functions

cube(n: float) -> float
This function takes a number as input and returns its cube.

Parameters:
n (float): The number to be cubed

Returns:
float: The cube of the input number

factorial(n: int) -> int
This function takes a number as input and returns its factorial.

Parameters:
n (int): The number to be factorialized

Returns:
int: The factorial of the input number

square(n: float) -> float
This function takes a number as input and returns its square.

Parameters:
n (float): The number to be squared

Returns:
float: The square of the input number

Problem 4: Attendance Management Module

Task:

1. Create a module attendance.py with functions:

o mark_present(student)

o mark_absent(student)

o get_attendance(student)

2. Add proper docstrings.

3. Generate and view documentation in terminal and browse

```python
def mark_present(student: str) -> None:
    """
    Marks a student as present in the attendance record.

    Args:
        student (str): The name of the student to be marked as present.
    """
    print(f"{student} is marked as present.")
def mark_absent(student: str) -> None:
    """
    Marks a student as absent in the attendance record.
    Args:
        student (str): The name of the student to be marked as absent.
    """
    print(f"{student} is marked as absent.")
def get_attendance(student: str) -> str:
    """Retrieves the attendance status of a student.
    Args:
        student (str): The name of the student whose attendance status is to be retrieved.
    Returns:
        str: The attendance status of the student (e.g., "Present", "Absent").
    """
    # For demonstration purposes, we will return a dummy attendance status.
    # In a real implementation, this would retrieve the actual attendance status from a database or record.
    return "Present"  # or "Absent" based on the actual attendance record
```

```
PS C:\Users\HARI PRIYA\Desktop\AIAC> & "C:\Users\HARI PRIYA\AppData\Local\Microsoft\WindowsApps\python3.11.exe" "c:/Users/HARI PRIYA/Desktop/AIAC/attendance.py"
PS C:\Users\HARI PRIYA\Desktop\AIAC> python -m pydoc attendance
Help on module attendance:

NAME
    attendance

FUNCTIONS
    get_attendance(student: str) -> str
        Retrieves the attendance status of a student.
        Args:
            student (str): The name of the student whose attendance status is to be retrieved.
        Returns:
            str: The attendance status of the student (e.g., "Present", "Absent").

    mark_absent(student: str) -> None
        Marks a student as absent in the attendance record.
        Args:
            student (str): The name of the student to be marked as absent.

    mark_present(student: str) -> None
        Marks a student as present in the attendance record.

        Args:
            student (str): The name of the student to be marked as present.

FILE
    c:\users\hari priya\desktop\aiac\attendance.py
```

index
attendance c:\users\hari priya\desktop\aiac\attendance.py

Functions

get_attendance(student: str) -> str
Retrieves the attendance status of a student.
Args:
student (str): The name of the student whose attendance status is to be retrieved.
Returns:
str: The attendance status of the student (e.g., "Present", "Absent").

mark_absent(student: str) -> None
Marks a student as absent in the attendance record.
Args:
student (str): The name of the student to be marked as absent.

mark_present(student: str) -> None
Marks a student as present in the attendance record.

Args:
student (str): The name of the student to be marked as present.

Problem 5: File Handling Function

Consider the function:

def read_file(filename):

with open(filename, 'r') as f:

return f.read()

Task:

1. Write documentation using all three formats.

2. Identify which style best explains exception

handling.

3. Justify your recommendation.

```python
def read_file(filename):
    """
    This function reads the content of a file and returns it as a string.

    Parameters:
    filename (str): The name of the file to be read

    Returns:
    str: The content of the file
    """
    with open(filename, 'r') as f:
        return f.read()

def read_file(filename):
    # This function reads the content of a file and returns it as a string.
    with open(filename, 'r') as f:
        return f.read()

def read_file(filename):
    """
    Reads the content of a file and returns it as a string.

    Args:
        filename (str): The name of the file to be read.
    Returns:
        str: The content of the file.
    Raises:
        FileNotFoundError: If the specified file does not exist.
        IOError: If there is an error reading the file.
    """
    with open(filename, 'r') as f:
        return f.read()
```

```
PS C:\Users\HARI PRIYA\Desktop\AIAC> & "C:\Users\HARI PRIYA\AppData\Local\Microsoft\WindowsApps\python3.11.exe" "c:/Users/HARI PRIYA/Desktop/AIAC/Handling.py"
PS C:\Users\HARI PRIYA\Desktop\AIAC> python -m pydoc Handling
Help on module Handling:

NAME
    Handling

FUNCTIONS
    read_file(filename)
        Reads the content of a file and returns it as a string.

        Args:
            filename (str): The name of the file to be read.
        Returns:
            str: The content of the file.
        Raises:
            FileNotFoundError: If the specified file does not exist.
            IOError: If there is an error reading the file.

        Raises:
            FileNotFoundError: If the specified file does not exist.
            IOError: If there is an error reading the file.

            FileNotFoundError: If the specified file does not exist.
            IOError: If there is an error reading the file.

            IOError: If there is an error reading the file.


FILE
FILE
    c:\users\hari priya\desktop\aiac\handling.py
```

It clearly mentions possible errors like FileNotFoundError and IOError.
This helps developers understand what can go wrong while reading a file. Inline style does not mention exceptions, and basic docstring style does not clearly specify errors.
For reliable and safe code, especially file handling, Google style is more informative and professional